GRADE 1 · INTRO DEBUGGING

Put It Back in Order

Fix the bug by re-ordering the blocks

Name: ..... Date: .....

★ **I can fix a program by putting the blocks back in order.**

Wrong order

Sometimes the blocks are all right but in the WRONG ORDER. To fix it, put them back in the right order. Remember: a program must BEGIN with the start block.

Bit's number path

Program 1 — target is 4

1 What is wrong?

Look at Program 1. Why can't Bit run it? (Hint: where is the start block?)

_____

2 Put it in order

Write the blocks in the right order so the program begins with the start block. Then run it — what number does Bit land on?

_____

_____

Program 2 — target is 4

start at 3

forward 3

back 2

3 You try — swap to fix

Run Program 2 from 3. Forward 3 passes the end, so Bit stops on 5, then back 2 lands on 3 — not 4. Swap the two move blocks and run again. Where does Bit land now?

Big idea

Code runs in order. If the blocks are in the wrong order, the program does the wrong thing — even when every block is right. Re-order them to fix the bug.

TEACHER GUIDE · GRADE 1 · INTRO DEBUGGING

Put It Back in Order

Quick facilitation notes & answer key

What this worksheet teaches

Students fix bugs by re-ordering blocks: first by moving the start block to the top (a program must begin with the start block), then by swapping two move blocks whose order changes the result. Fixing a program by changing the order is a kind of debugging. No coding experience needed.

Before you start (no computer needed)

No computer — paper and a pencil. You read each prompt aloud. Optional: tape a number line 0 to 5 on the floor and mark the target, so students can walk the buggy program and feel Bit miss.

How to lead it (about 20 minutes)

1. Show them (you do). A program must begin with the start block — show one where it is in the middle and cannot run.

2. Do one together. Exercise 1 — the start block is in the middle, so the order is wrong; Exercise 2 — put it on top and run.

3. Let them try. Students swap two move blocks in Program 2 to reach the target (Ex 3).

Answer key (these are the verified correct answers)

Exercise 1 — the start block ("start at 1") is in the middle; a program must begin with the start block, so the order is wrong.

Exercise 2 — right order: start at 1, forward 2, forward 1. Run: 1 → 3 → 4. Bit lands on 4.

Exercise 3 — buggy order (forward 3, back 2): 3 → 5 (stops at the end) → 3. Swapped (back 2, forward 3): 3 → 1 → 4. Bit lands on 4.

If students get stuck — and ways to adjust

Watch for: changing a block's number instead of its order. Here the blocks are right — the order is wrong.

Make it easier: cut the blocks into cards and slide them into a new order.

Make it harder: ask why the swap in Exercise 3 changes the result (the end of the path).

Ontario curriculum link (official wording)

C3.1 — solve problems and create computational representations of mathematical situations by writing and executing code, including code that involves sequential events.

In plain terms: students follow (or write) step-by-step instructions, in order, to get a result.

C3.2 — read and alter existing code, including code that involves sequential events, and describe how changes to the code affect the outcomes.

In plain terms: students read a program, change a block to fix it, and explain what the change did.

You'll know it worked when

The child puts the start block on top and reorders the moves so Bit reaches the target.